

PowerShell Cheat Sheet

From basics → piping/filtering → system analysis → scripting & remote execution

NAVIGATE	READ	DISCOVER	FILTER	ANALYZE	REMOTE
cd / pwd / Get-ChildItem	Get-Content	Get-Command / Get-Help	Where-Object / Sort-Object	Get-Process / Get-Service / Get-NetTCPConnection	Invoke-Command

3 rules to remember

These unlock most room questions

Core syntax: Verb-Noun → Get-Content, Set-Location

Pipe: | sends output objects to the next command. PowerShell pipes objects, not only text.

Golden pipeline: Get → Filter → Sort → Select

What is PowerShell?

Shell + scripting language

PowerShell commands are called cmdlets. Their names usually follow Verb-Noun.

Aliases

Short names for commands

cat → Get-Content cd → Set-Location

dir / ls → Get-ChildItem echo → Write-Output

Think like a SOC analyst

Use commands to ask questions

What is running? → Get-Process

What services exist? → Get-Service

Who is talking? → Get-NetTCPConnection

Think like a scripter

Automate repeated work

Put commands in a script instead of repeating them manually. Useful for logs, IOC extraction, enumeration, integrity checks, and remote tasks.

01 - Navigation, Files & Command Discovery

<code>pwd</code>	Show current directory	<code>pwd</code>	Think: Where am I?
<code>cd PATH</code>	Change directory	<code>cd C:\Users\captain\Documents</code>	<code>cd</code> = change directory
<code>cd ..</code>	Go to parent folder	<code>cd ..</code>	<code>..</code> = parent; <code>.</code> = current folder
<code>Get-ChildItem</code>	List items in current folder	<code>Get-ChildItem</code>	Think: What is inside this folder? Aliases: <code>ls</code> , <code>dir</code>
<code>Get-Content FILE</code>	Read file content	<code>Get-Content notes.txt</code>	Think: What is inside this file? Alias: <code>cat</code>
<code>Get-Command</code>	List available commands	<code>Get-Command</code>	Use to discover what PowerShell can do
<code>Get-Command -Verb Remove</code>	Find commands with a specific verb	<code>Get-Command -Verb Remove</code>	Example: <code>Remove-Item</code> , <code>Remove-Job</code> , ...
<code>Get-Command -CommandType Function</code>	Filter commands by type	<code>Get-Command -CommandType Function</code>	Useful when the question names a command type
<code>Get-Help COMMAND</code>	Get command documentation	<code>Get-Help Get-Date</code>	Syntax + parameters + description
<code>Get-Help COMMAND -Examples</code>	Show practical examples	<code>Get-Help New-LocalUser -Examples</code>	Best first move when you know the cmdlet but not how to use it
<code>Get-Alias</code>	List aliases	<code>Get-Alias</code>	Use <code>Get-Alias cat</code> to see what <code>cat</code> maps to

File vs Folder - never mix them up

The most common beginner trap

`Get-ChildItem` → lists items inside a folder.

`Get-Content` → reads the content inside a file.

02 - Piping, Filtering, Sorting & Selecting

Pipe |

Output from the left becomes input to the right

`Get-ChildItem | Sort-Object Length`

Left: get file objects. Right: sort those objects by their Length property.

Key PowerShell idea: the pipe passes objects with properties and methods, so the next cmdlet can work with real fields such as Name, Length, Extension, etc.

Where-Object	Filter objects by a condition	<code>Get-ChildItem Where-Object -Property Extension -eq .txt</code>	Think: FILTER
-eq	Equal to	<code>... -Property Extension -eq .txt</code>	= / equal
-ne	Not equal to	<code>... -Property Extension -ne .txt</code>	Exclude matches
-gt	Greater than	<code>... -Property Length -gt 100</code>	100 itself is excluded
-ge	Greater than or equal	<code>... -Property Length -ge 100</code>	Includes 100
-lt	Less than	<code>... -Property Length -lt 100</code>	Strictly below 100
-le	Less than or equal	<code>... -Property Length -le 100</code>	Includes 100
-like	Pattern match	<code>... -Property Name -like ship*</code>	Useful with wildcards like *
Sort-Object	Sort by a property	<code>Get-ChildItem Sort-Object Length</code>	Default: ascending
<code>Sort-Object -Descending</code>	Reverse sort	<code>Get-ChildItem Sort-Object Length -Descending</code>	Largest first for Length
Select-Object	Choose properties or limit objects	<code>Get-ChildItem Select-Object Name,Length</code>	Think: SHOW ONLY WHAT I NEED
<code>Select-Object -First 1</code>	Return first object	<code>... Select-Object -First 1</code>	After descending sort = largest

Classic room answer

Largest file in current directory

`Get-ChildItem | Sort-Object Length -Descending | Select-Object -First 1`

Flow: Get files → sort biggest-to-smallest → take first.

03 - Search + Real-Time System Analysis

Select-String	Search text/patterns inside files	Select-String -Path .\captain-hat.txt -Pattern hat	Similar idea to grep / findstr. Supports regex.
Get-Process	List currently running processes	Get-Process	Useful for incident response, malware analysis, and troubleshooting
Get-Process -Id PID	Look up one process by PID	Get-Process -Id 1444	Use after finding a PID elsewhere
Get-Service	List Windows services and status	Get-Service	Key fields: Status, Name, DisplayName
Get-NetTCPConnection	List current TCP connections	Get-NetTCPConnection	Key fields: Local/Remote address & port, State, OwningProcess
Get-FileHash -Path FILE	Generate file hash	Get-FileHash -Path .\ship-flag.txt	Example uses SHA256; useful for integrity/tampering checks
Get-Item -Path FILE -Stream *	Show NTFS streams	Get-Item -Path .\file.txt -Stream *	Named extra stream = ADS; :\$DATA is the default stream

Network triage pattern

Map a connection to its process

Get-NetTCPConnection

Read OwningProcess → that is the PID.

Get-Process -Id 1444

Now you know which process owns the connection.

Service investigation pattern

Find a tampered service

Get-LocalUser | Select-Object Name,Description

Read the shady user's Description (their motto).

Get-Service | Where-Object DisplayName -eq <motto>

Return Name — that is the service name the question usually asks for.

Hash = file fingerprint

Integrity / IR / malware analysis

Same content → same hash (with the same algorithm). Changed content → hash changes.

Default room example: SHA256

ADS in one picture

Extra named stream attached to an NTFS file

house_log.txt:housinginfo

:\$DATA = normal/default data stream; housinginfo = ADS.

04 - Scripting & Remote Execution

Scripting

Automate a sequence of commands

A script is a text file containing commands that run automatically. This saves time, reduces manual mistakes, and handles repetitive or complex tasks.

<code>Invoke-Command -ComputerName PC -ScriptBlock { command }</code>	Run a command on a remote computer	<code>Invoke-Command -ComputerName RoyalFortune -ScriptBlock { Get-Service }</code>	ComputerName = WHERE; ScriptBlock = WHAT
<code>Invoke-Command -FilePath SCRIPT.ps1 -ComputerName PC</code>	Run a local script file remotely	<code>Invoke-Command -FilePath C:\scripts\test.ps1 -ComputerName Server01</code>	Use when the work is already packaged in a .ps1 file
<code>-Credential</code>	Specify credentials for a remote connection	<code>Invoke-Command -ComputerName Server01 -Credential Domain01\User01 -ScriptBlock { Get-Culture }</code>	If the question says credentials are not needed, leave this out

Blue Team

Defensive use

Automate log analysis, anomaly detection, IOC extraction, malware analysis, and system scans.

Red Team

Offensive / testing use

Automate enumeration, remote commands, and attack simulations during authorized engagements.

System Administration

Operational use

Automate integrity checks, configuration, policy enforcement, health monitoring, and incident response.

Security mindset

PowerShell is a tool, not a verdict

powershell.exe ≠ automatically malicious. Judge the command, parent process, destination, context, and behavior.

Remote command mental model

Memorize this sentence

`Invoke-Command = Run -ComputerName = Where -ScriptBlock = What`

05 - One-Page Exam Map

I need to know where I am	<code>pwd</code>	I need largest first	<code>... Sort-Object Property -Descending</code>
I need to move folders	<code>cd PATH</code>	I only need certain fields	<code>... Select-Object Name,Length</code>
I need to list a folder	<code>Get-ChildItem</code>	I need the first result	<code>... Select-Object -First 1</code>
I need to read a file	<code>Get-Content file.txt</code>	I need to search text in files	<code>Select-String -Path FILE -Pattern TEXT</code>
I need to discover commands	<code>Get-Command</code>	I need running processes	<code>Get-Process</code>
I need commands with a specific verb	<code>Get-Command -Verb Remove</code>	I need a process by PID	<code>Get-Process -Id PID</code>
I need help/examples	<code>Get-Help COMMAND -Examples</code>	I need Windows services	<code>Get-Service</code>
I need an alias	<code>Get-Alias</code>	I need TCP connections	<code>Get-NetTCPConnection</code>
I need to filter	<code>... Where-Object ...</code>	I need the process behind a TCP connection	<code>Read OwingProcess → Get-Process -Id PID</code>
I need greater-than logic	<code>... -gt VALUE</code>	I need a file fingerprint	<code>Get-FileHash -Path FILE</code>
I need pattern matching	<code>... -like PATTERN</code>	I need ADS	<code>Get-Item -Path FILE -Stream *</code>
I need sorting	<code>... Sort-Object Property</code>	I need a remote command	<code>Invoke-Command -ComputerName PC -ScriptBlock { command }</code>

Last-minute memory

The shortest useful mental map

Get = retrieve | Set = change | New = create | Remove = delete

Where-Object = filter | Sort-Object = sort | Select-Object = choose

Get-Process = running | Get-Service = services | Get-NetTCPConnection = network